Part 1: Lexical Definitions for the Different Tokens (REGEX)

Identifiers and Literals	
ID	*(Regex)* **[a-zA-Z][a-zA-Z0-9_]*** ***Notes:*** *Rules for Identifiers are…* 1. *The first character must be an uppercase or lowercase English letter (a-z or A-Z). Digits or underscores are not allowed here.* 2. *The identifier must contain at least one character. After the first character, there may be zero or more characters, which may be letters (a-z or A-Z), digits (0-9), or underscores.* 3. *All keywords listed below are considered reserved words and therefore cannot be used as valid identifiers.*
LT_INT	*(Regex)* '**-**'? **[0-9]+** ***Notes:*** *Integer literals should be a non-empty sequence of digits. A negative sign is optional to denote negative integers.*
LT_FLOAT	*(Regex)* '**-**'? **[0-9]+** '**.**' **[0-9]+** ***Notes:*** *Float literals should be a non-empty sequence of digits, followed by a decimal point, then followed by another non-empty sequence of digits. A negative sign is optional to denote negative floats.*
LT_STR	*(Regex)* '"' (~["**\\\r\n**] | '****'.)* '"'
LT_CHAR	*(Regex)* '****' ('****'. | ~['**\\\r\n**]) '****'
LT_BOOL	**zhen** | **buzhen** | 真 | 不真
NULL	**meiyou** | 没有 | 沒有
CONST	**yongyuan** | 永远 | 永遠

Control Flow	
IF	**ruguo** | 如果
ELSE	**xiayige** | 下一个 | 下一個
ELIF	**xiayigeruguo** | 下一个如果 | 下一個如果
WHILE	**buduanruguo** | 不断如果 | 不斷如果
REPEAT	**laiyibian** | 来一遍 | 來一遍
UNTIL	**dao** | 到
FOR	**buduan** | 不断 | 不斷
RET	**huijia** | 回家
BREAK	**ting** | 停
CONT	**buting** | 不停

Datatypes and Variables	
KW_INT	**hao** | 号 | 號
KW_FLOAT	**erhao** | 二号 | 二號
KW_CHAR	**pinyin** | 拼音
KW_STR	**duopinyin** | 多拼音
KW_BOOL	**zhende** | 真的
KW_RECORD	**yinyue** | 音乐 | 音樂
KW_MAIN	**main** | **zhuyao** | 主要
TYPEDEF	**yisi** | 意思
VOID	**gan** | 干 | 乾

Arithmetic Operators	
ADD	**jia** | **+** | 加
SUB	**fu** | **-** | 负 | 負
MULT	**duojia** | ***** | 多加
DIV	**duofu** | **/** | 多负 | 多負
MOD	**xialai** | **%** | 下来 | 下來
ASSIGN	**shi** | **=** | 是
EQ	**zhege** | **==** | 这个 | 這個
NEQ	**buzhege** | **!=** | 不这个 | 不這個
LT	**xiao** | **<** | 小
GT	**duo** | **>** | 多
LTE	**xiaozhege** | **<=** | 小这个 | 小這個
GTE	**duozhege** | **>=** | 多这个 | 多這個
INC	**jiajia** | **++** | 加加
DEC	**fufu** | **--** | 负负 | 負負

Logical Operators	
AND	**HE** | **&&** | 和
OR	**HAI** | **||** | 还 | 還
NOT	**LING** | **!** | 零

Delimiters and Symbols	
LPAR	**(**
RPAR	**)**
LBAR	**[**
RBAR	**]**
LBRACE	**{**
RBRACE	**}**
SEMICOLON	**;**
COMMA	**,**
DOT	**.**
REF	**&**
DE_REF	**^**

Comments ***Note:*** *These tokens are skipped entirely and are not processed by the succeeding parser.*	
SINGLE_COMMENT	*(Regex)* '**//**' ~[**\r\n**]*
MULTI_COMMENT	*(Regex)* '**/***' .*? '***/**'

Input and Output	
INPUT	**geiwo** | 给我 | 給我
PRINT	**geini** | 给你 | 給你

Error Literals	
ERROR_FLOAT	*(Regex)* '**-**'? [**0**-**9**]+ '**.**' [**0**-**9**]+ ('**.**' [**0**-**9**]+)+
ERROR_STRING	*(Regex)* " (~["**\\\r\n;}**])* **EOF**?
ERROR_CHAR	*(Regex)* '****" (~['**\r\n;}**])*
ERROR_CHAR_2	*(Regex)* '****" ('****' . | ~['**\\\r\n**]) ('****' . | ~['**\\\r\n**])+ '****"
ERROR_ID	*(Regex)* [**0**-**9**_][**a**-**zA**-**Z0**-**9**_]*
ERROR_MULTI_COMMENT	*(Regex)* '**/***' (~'*****' | '*****' ~'/')* **EOF**
ERROR_ANY	*(Regex)* .

End-of-File Token	
EOF	**EOF**

Part 2: Context-Free Grammar for Our Proposed Language

The grammar G = (*V*, *T*, *S*, *P*) of our proposed language is defined by:

V =
{
<program>, <mainList>, <globalDecl>, <mainFunc>, <typedefItem>, <panicMode>,
<constDecl>, <varDecl>, <recordDecl>, <varDeclFor>, <dataType>, <recordContent>,
<funcDecl>, <funcRet>, <funcVoid>, <paramList>, <funcContent>, <funcCall>, <argList>,
<statement>, <assignStat>, <assignStatNoEnd>, <returnStat>, <assignableVal>,
<ifStat>, <elifList>, <forStat>, <whileStat>, <repeatUntilStat>,
<expression>, <arrayInitExp>, <expList>, <nullExp>, <refVal>, <refValPrime>, <deRefVal>,
<idWrapper>, <floatWrapper>, <stringWrapper>, <charWrapper>,
<numExp>, <numExp1>, <numExp2>, <numRef>,
<strExp>, <strBase>, <strVal>,
<boolExp>, <boolExp1>, <boolExp2>, <boolExp3>, <boolExp4>, <boolVal>
}

T =
{
ID, LT_INT, LT_FLOAT, LT_STR, LT_CHAR, LT_BOOL, NULL, CONST,
IF, ELSE, ELIF, WHILE, REPEAT, UNTIL, FOR, RET, BREAK, CONT,
KW_INT, KW_FLOAT, KW_CHAR, KW_STR, KW_BOOL, KW_RECORD, KW_MAIN, TYPEDEF, VOID,
ADD, SUB, MULT, DIV, MOD, ASSIGN, EQ, NEQ, LT, GT, LTE, GTE, INC, DEC,
AND, OR, NOT,
LPAR, RPAR, LBAR, RBAR, LBRACE, RBRACE, SEMICOLON, COMMA, DOT, REF, DE_REF,
INPUT, PRINT,
ERROR_ID, ERROR_FLOAT, ERROR_STRING, ERROR_CHAR, ERROR_CHAR_2, ERROR_ANY,
EOF
}

S = <program>

***P* has the following productions, expressed in Extended Backus-Naur Form (EBNF)...**
*Note: Text in **bold** indicates **terminal symbols** (as defined as **tokens** in **Part 1**), while text not in bold represents non-terminals.*

Productions for the Program		
<program>	→	<mainList> <mainFunc> **EOF**
<mainList>	→	(<typedefItem> | <globalDecl> | <panicMode>)*
<globalDecl>	→	<funcDecl> | <constDecl> | <recordDecl> | <varDecl>
<mainFunc>	→	(<dataType> | VOID) **KW_MAIN LPAR** <paramList> **RPAR LBRACE** <funcContent> **RBRACE**
<typedefItem>	→	**TYPEDEF** (<dataType> | <recordDecl>) <idWrapper> **SEMICOLON**

Productions for Using Declarations		
<constDecl>	→	**CONST** <dataType> <idWrapper> (**ASSIGN** <expression>)? **SEMICOLON**
<varDecl>	→	<dataType> <idWrapper> (ASSIGN <expression>)? **SEMICOLON**
<recordDecl>	→	**KW_RECORD** <idWrapper>? **LBRACE** <recordContent> **RBRACE SEMICOLON**
<varDeclFor>	→	<dataType> <idWrapper> (ASSIGN <expression>)? **SEMICOLON**
<dataType>	→	**KW_INT** | **KW_FLOAT** | **KW_CHAR** | **KW_STR** | **KW_BOOL** | <recordDecl> | <idWrapper> | <dataType> **DE_REF** | <dataType> **LBAR LT_INT RBAR**
<recordContent>	→	(<varDecl> | <recordDecl>)*

Productions for Functions		
<funcDecl>	→	<funcRet> | <funcVoid>
<funcRet>	→	<dataType> <idWrapper> **LPAR** <paramList> **RPAR LBRACE** <funcContent> **RBRACE**
<funcVoid>	→	**VOID** <idWrapper> **LPAR** <paramList> **RPAR LBRACE** <funcContent> **RBRACE**
<paramList>	→	(<dataType> <idWrapper> (**COMMA** <dataType> <idWrapper>)*)?
<funcContent>	→	(<statement> | <panicMode>)*

Productions for Statements		
<statement>	→	<expression> **SEMICOLON** | <funcCall> **SEMICOLON** | <assignStat> | <ifStat> | <whileStat> | <repeatUntilStat> | <forStat> | <returnStat> | **LBRACE** <statement>* **RBRACE** | **PRINT LPAR** <expression> **RPAR SEMICOLON** | **INPUT LPAR** <assignableVal> (**COMMA** <assignableVal>)* **RPAR SEMICOLON** | **BREAK SEMICOLON** | **CONT SEMICOLON** | <varDecl> | <constDecl>
<assignStat>	→	<assignableVal> **ASSIGN** <expression> **SEMICOLON**
<assignStatNoEnd>	→	<assignableVal> **ASSIGN** <expression>
<returnStat>	→	**RET** <expression>? **SEMICOLON**

Productions for Control Flow		
<ifStat>	→	**IF LPAR** <expression> **RPAR** <statement> <elifList>?
<elifList>	→	**ELIF LPAR** <expression> **RPAR** <statement> <elifList>? | **ELSE** <statement>
<forStat>	→	**FOR LPAR** <varDeclFor> <expression> **SEMICOLON** <assignStatNoEnd> **RPAR** <statement> | **FOR LPAR** <assignStat> <expression> **SEMICOLON** <assignStatNoEnd> **RPAR** <statement>
<whileStat>	→	**WHILE LPAR** <expression> **RPAR** <statement>
<repeatUntilStat>	→	**REPEAT** <statement> **UNTIL LPAR** <expression> **RPAR SEMICOLON**

Productions for General Expressions		
<expression>	→	<numExp> | <strExp> | <boolExp> | <funcCall> | <deRefVal> | <arrayInitExp> | <nullExp>
<arrayInitExp>	→	**LBRACE** <expList> **RBRACE**
<expList>	→	(<expression> (**COMMA** <expression>)*)?
<nullExp>	→	**NULL**
<refVal>	→	<idWrapper> <refValPrime>
<refValPrime>	→	**LBAR** <numExp> **RBAR** <refValPrime> | **DOT** <refVal> <refValPrime> | **ε**
<deRefVal>	→	**DE_REF** <deRefVal> | **DE_REF** <refVal>
<assignableVal>	→	<deRefVal> | <refVal>

Productions for Function Calls		
<funcCall>	→	<idWrapper> **LPAR** <argList> **RPAR**
<argList>	→	(<expression> (**COMMA** <expression>)*)?

Productions for ID Wrappers		
<idWrapper>	→	**ID** | **ERROR_ID**
<floatWrapper>	→	**LT_FLOAT** | **ERROR_FLOAT**
<stringWrapper>	→	**LT_STR** | **ERROR_STRING**
<charWrapper>	→	**LT_CHAR** | **ERROR_CHAR** | **ERROR_CHAR_2**

Productions for Numerical Expressions		
<numExp>	→	<numExp1> ((**ADD** | **SUB**) <numExp1>)*
<numExp1>	→	<numExp2> ((**MULT** | **DIV** | **MOD**) <numExp2>)*
<numExp2>	→	**LPAR** <numExp> **RPAR** | <charWrapper> | **LT_INT** | <floatWrapper> | **INC** <numRef> | **DEC** <numRef> | <numRef> **INC** | <numRef> **DEC** | <numRef> | **REF** <refVal> | <deRefVal> | <funcCall>
<numRef>	→	<refVal>

Productions for String Expressions		
<strExp>	→	<strBase> (**ADD** <strBase>)*
<strBase>	→	<strVal> | <stringWrapper> | <charWrapper> | <funcCall> | <deRefVal> | **LPAR** <strExp> **RPAR**
<strVal>	→	<refVal>

Productions for Boolean Expressions		
<boolExp>	→	<boolExp1> (**OR** <boolExp1>)*
<boolExp1>	→	<boolExp2> (**AND** <boolExp2>)*
<boolExp2>	→	(<numExp> | <nullExp>) (**EQ** | **NEQ**) (<numExp>|<nullExp>) | (<strExp> | <nullExp>) (**EQ** | **NEQ**) (<strExp> |<nullExp>) | <boolExp3> ((**EQ** | **NEQ**) <boolExp3>)*
<boolExp3>	→	<numExp> (**LT** | **GT** | **LTE** | **GTE**) <numExp> | | <strExp> (**LT** | **GT** | **LTE** | **GTE**) <strExp> | <boolExp4>
<boolExp4>	→	**LPAR** <boolExp> **RPAR** | **NOT** <boolExp4> | <boolVal> | **LT_BOOL** | <funcCall> | <deRefVal>
<boolVal>	→	<refVal>

Productions for Error Detection		
<panicMode>	→	~(**RBRACE** | **EOF** | **KW_MAIN**) **ERROR_ANY***? **SEMICOLON**